PRIVOKE: A Client-Side Semantic Privacy Firewall for User Prompts in Web-Hosted LLMs

Nicholas Bisset*
Monash University Clayton, Australia
nbis0006@student.monash.edu

Manpreet Sahadevan, Nixon Ngai, and Sanoop Mallissery
Monash University Malaysia, Malaysia
{msah0018, nnga0004}@student.monash.edu, sanoop.mallissery@monash.edu

***Abstract*—Useful prompts often contain context that a user may not intend to disclose to an external language-model provider. PRIVOKE places local inspection and explicit allow, warn or block actions before transmission on supported client paths. Its lifecycle connects request enforcement to a separate training service, which checks version and retention constraints before distributing accepted parameters; independently randomized telemetry supplies no training data. We evaluate detection and enforcement, retained behavior during updates, and systems and privacy overhead. On 1,560 authored provisional cases, updated packages improved contextual policy agreement, while targeted detector removal exposed a tradeoff between unnecessary restrictions and missed protection. A frozen 498-row semantic annotation-presence check found specificity of only 13.36 to 21.76%. Thirty-three installed receiver probes verified declared request behavior. Every matched full-model candidate improved aggregate agreement yet failed retention. Browser measurements quantified rules/NER decision cost without isolating incremental overhead; telemetry audits found a small excess over nominal privacy loss and substantial synthetic estimation error. These findings support an inspectable protection and revision mechanism while leaving independent contextual accuracy, useful adaptation without lost protection and practical user benefit unresolved.**

***Index Terms*—Prompt privacy, local inference, request enforcement, model regression, randomized telemetry, local differential privacy.**

I. INTRODUCTION

Web-hosted large language models support writing, coding and analysis by processing the instructions and contextual material that users provide [1]–[3]. A useful prompt may therefore contain much more than a question: a user seeking help with a report might include a colleague's details, health information or confidential organizational material. The same context that helps the model complete the task can consequently disclose information to an external provider. Reported workplace incidents illustrate how this can occur during otherwise legitimate work [4], [5]. Controls over subsequent storage or use do not prevent the initial transfer. Inspecting the request on the user's device provides an opportunity to intervene while the information is still under local control.

Deciding when to intervene requires more than recognizing names or identifiers. Two prompts can discuss the same subject while differing in whose information they contain, who is intended to receive it and whether that disclosure is appropriate. Nissenbaum's account of contextual integrity similarly relates privacy to the norms governing information flows within a particular context [6]. A detector can represent some of these distinctions, but its labels do not independently establish authorization. Its decision must also have an observable effect on the request: notifying a user that content may be sensitive is different from preventing that content from leaving the device. These considerations motivate evaluating contextual predictions, policy decisions and actual transmission separately.

Motivated by this problem, we present PRIVOKE, an implemented client-side privacy firewall for supported outgoing prompts. The local detector combines rules, named entity recognition and semantic classification, describing information through sensitivity, intended audience and categories before selecting ALLOW, WARN or BLOCK. The extension releases the original request for ALLOW and WARN and cancels a supported request for BLOCK. WARN attempts to display a notice without waiting for acknowledgement, so it does not establish consent or prevented disclosure. This explicit request behavior allows the evaluation to distinguish a correct classification from an intervention that actually stops transmission.

Maintaining this protection introduces a second problem: a revised model can improve average accuracy while becoming wrong on cases that its predecessor handled correctly [7]. PriVoke therefore separates training a candidate from accepting it for use. A server-side runtime learns from supplied labelled examples, and the update service checks the candidate against its exact parent model and declared retention criteria before publication. Clients verify accepted artifacts and refresh their local parameters without an application release. A third, independent path randomizes coarse event reports on the workstation. Neither intercepted prompts nor telemetry supplies training labels, allowing detector maintenance and reporting to be examined without treating collected user traffic as supervision.

Local protection and learned sanitization already have substantial precedent in Casper, Hide and Seek and ProSan [8]–[11]. Our contribution is the design and evaluation of a lifecycle that connects explicit request enforcement, guarded detector revision and separate randomized reporting. We examine whether the implemented mechanisms deliver their intended behavior, how improvements coexist with casewise losses, and what processing and reporting costs remain. This

*Corresponding author: Nicholas Bisset.
GitHub Repository Link

scope supports an engineering contribution even when an update is rejected or an evaluation exposes a limitation; it does not presume that local execution, successful training or a passing guard establishes effective privacy protection.

We organize the evaluation around three questions:

1) **RQ1: Does PriVoke detect and enforce privacy-sensitive prompt policies effectively?**
2) **RQ2: Can PriVoke update its detector without sacrificing previously protected behavior?**
3) **RQ3: What systems and privacy overhead does PriVoke introduce?**

RQ1 connects contextual predictions to the consequences of allowing or stopping a request. RQ2 asks whether improvements survive a retention requirement, rather than assuming that a newer model deserves publication. RQ3 examines the resources consumed and information released by protection itself. The following sections develop the prior-work basis and system design before answering these questions using the available evidence, including negative results and the limits of provisional contextual labels.

II. Related Work and Contribution

Local intervention and contextual decisions. Casper demonstrates how prompt protection can be placed on the user's device through a browser extension combining rules, entity recognition and local topic analysis [8]. Its reviewed prototype replaces identifiers with placeholders, supports user-triggered restoration in responses and requires acknowledgement of topic warnings. Analysis is initiated through a keyboard shortcut. Casper also permits custom rules and topics and compatible model replacement, establishing precedent for both layered local inspection and configurable protection. PriVoke builds on that motivation while examining a different request contract: its supported hooks automatically apply a decision, but a WARN releases unchanged content without waiting for the user.

Hide and Seek (HaS) addresses the usefulness of protected interactions by hiding entities before cloud inference and recovering them in the response [10]. Its generative variant trains hiding and recovery models, while the label-based variant uses an entity recognizer for hiding. ProSan instead uses task-importance and privacy proxies to select word replacements, then trains a sanitizer for local execution [11]. These approaches show that learned local protection can include both transformation and recovery. PromptShield considers a related enterprise setting through role/policy classification [12]. Together, these works motivate separating the detection of sensitive information from the decision about its disclosure and the user's ability to complete a task. PriVoke's sensitivity, audience and category labels provide a representation for its policy; they do not constitute an independently validated model of contextual authorization.

Retaining behavior during model revision. The distinction between better average performance and preserved individual predictions is established in model-update research. Yan et al. study *negative flips*, where an updated image classifier becomes incorrect on a previously correct example, and introduce positive-congruent training to reduce such regressions [7]. Their result motivates measuring both gains and losses, although image-classification evidence does not establish privacy-policy correctness. PriVoke examines the related engineering decision of whether to publish a candidate: label and action outcomes are compared with the declared base, and a configured retention guard can reject an otherwise improved model. Binding that decision to a versioned artifact and verified client refresh makes acceptance distinct from successful optimization. The contribution is this implemented and evaluated connection; neither casewise regression analysis nor model training is claimed as new, and the guard covers only its declared cases and criteria.

Privacy of operational reporting. Local randomization provides a foundation for collecting aggregate information without sending each underlying value directly to a collector. RAPPOR applies randomized response to client-side statistics and explicitly addresses the additional privacy cost of repeated reporting [13]. PriVoke uses categorical event reports with a local budget ledger, rather than adopting RAPPOR's memoized reporting mechanism. Its relevant question is therefore whether the implemented sampler and accounting satisfy their stated event-content bounds, and whether the resulting estimates are useful. We examine finite arithmetic, estimation error and contribution selection separately. The reporting mechanism does not protect participation or network metadata, and aggregate event fields supply neither corrected contextual labels nor evidence that the detector should be retrained.

Contribution and comparison boundaries. These strands lead to three distinct evaluation needs: establish the relationship between a detection and its request outcome; assess a revision through retained behavior as well as aggregate gain; and account for the resources and disclosure introduced by the protection mechanism. PriVoke brings them together in one implemented lifecycle. It does not replace outgoing text with its returned masked spans or recover response entities, so sanitization utility cannot be inferred from its request decisions. Likewise, Casper's entity/topic measures, HaS's whole-text recovery resistance and ProSan's item-replacement and task-utility measures have different targets and denominators. The resulting contribution is a measured account of enforcement, retention and reporting tradeoffs, with broader contextual accuracy and practical benefit left to evidence that the present studies do not supply.

III. System Design and Privacy Boundary

A. Scope and Trust

PriVoke is designed to intervene when personal, third-party or organizational information is about to leave the workstation through a supported prompt request. Its protection depends on the enabled extension, local runtime and supervisor, workstation and authenticated model publisher operating as configured. These components form the trusted boundary. A compromised client, malicious publisher or page that deliberately replaces the interception hooks can undermine

that boundary, while information already held by a provider cannot be protected by stopping a later outgoing request.

The implemented interception path recognizes prompt POST formats submitted through page `fetch` and asynchronous `XMLHttpRequest`. Workers, attachments, other transports and changed endpoints require their own validation; support for the two hooks does not establish coverage of those paths. Users can also disable protection, which bypasses inspection by design. An optional hosted-inference configuration sends text beyond the workstation and therefore has a different disclosure boundary. It is excluded from the local-inference measurements in this paper.

Within this boundary, Figure 1 separates three kinds of information flow. Prompt text travels from the browser to the workstation detector and reaches the hosted LLM only if the request action permits release. Supplied labelled examples follow a separate training path, from which accepted model parameters return to clients. Coarse event fields follow a third path through local randomization to a telemetry collector. The separation makes each service's input explicit: maintaining the detector does not require collecting intercepted prompts, and the collector supplies no training labels. Whether the detector makes an appropriate decision remains an empirical question.

B. Local Inspection and Request Enforcement

Inspection begins when a page hook extracts the prompt from a supported outgoing request. The extension's background process sends the text through a loopback gRPC-Web bridge to the workstation's Python runtime. Native messaging starts the supervisor that manages the bridge and runtime; it does not send the prompt to the separate server-side training runtime. After analysis, the hook uses the returned action to release or cancel the original request. The training instance is not a remote fallback for local user inference.

The runtime first normalizes the text while retaining offsets that connect its findings to the original input. It then evaluates regular-expression rules before the enabled named entity recognition (NER) and semantic layers. A rule-based BLOCK ends the cascade immediately. Otherwise, the findings are combined using the strongest action. This ordering has a direct consequence for interpretation: a semantic classifier cannot reverse an earlier rule-based restriction, and an early BLOCK means that the later requested detectors did not run.

Failure handling follows the same need to identify what was actually assessed. A failed requested layer changes an otherwise ALLOW result to BLOCK, while valid non-ALLOW findings remain visible. If analysis itself is unavailable, the extension blocks supported requests. Losing the parameter stream can instead leave rules and NER available as a reduced detector configuration. That fallback may release a request despite losing semantic coverage, so a returned action must be interpreted together with the recorded layer execution.

The semantic classifier represents a prompt through sensitivity, audience and information categories. Sensitivity distinguishes no identified risk (S0), low-risk personal context (S1), potentially harmful personal information (S2), and highly sensitive information, identifiers or secrets (S3). Audience, called *visibility* in the implementation, comprises public (P0), semi-public (P1), authentication-restricted (P2), group-private (P3), personal-private (P4) and unspecified (PU) contexts. The ten categories are health, politics, religion, criminal, financial, sexual, child, location, identity and third-party information; several may apply together.

The policy normally assigns BLOCK to S3 and WARN to S2. S0/S1 normally allow, except that identity or location triggers WARN when both categories occur together or visibility is P2–P4. S0 with no categories always allows. A numeric confidence below 0.5 overrides this mapping: S3 becomes WARN, S2 remains WARN only with identity or location, and all other cases become ALLOW. Confidence summarizes sensitivity/category heads and is not a calibrated probability of joint correctness or authorized disclosure. Neither the labels nor this deterministic mapping has been independently validated as a model of contextual authorization.

The models performing this task are classifiers, not conversational response generators. The evaluation compares historical small classifiers with later packages retaining pretrained BERT encoders and poolers [14]–[17]. Both generations offer efficient, balanced and quality packages. These names identify configurations rather than a guaranteed performance ordering. Tokenizers, capacities, thresholds and training histories differ as well, so the comparison measures the resulting package change rather than the isolated effect of pretraining or architecture.

C. Guarded Detector Updates

The update path asks two separate questions: whether a candidate belongs to the intended model, and whether its behavior meets the configured acceptance criteria. A controller prepares bounded labelled batches and held-out guard cases, excluding protected rows, texts and groups from training. The server runtime loads the specified base, trains a candidate and returns parameter changes and metrics tied to that base. Before publication, the updater checks base identity, permitted tensors and shapes, finite values, magnitude limits and the strategy's configured guards. Artifact checks establish that the candidate is well formed; retention checks assess its behavior on declared cases. Completing a fit or transferring its parameters cannot substitute for either decision.

An accepted candidate is published through an atomic compare-and-swap operation, which requires the current base still to match its declared parent. This prevents a decision made for one model from silently replacing a different intervening revision. Durable replay receipts associate an identical retry with its earlier artifact outcome instead of applying it twice. Head and underlying-model stages are separate publications, however, so rejecting a later stage does not undo an accepted head update. Clients receive versioned parameters from the streaming service and verify artifact identities before refreshing cached weights, allowing model delivery without an application release.

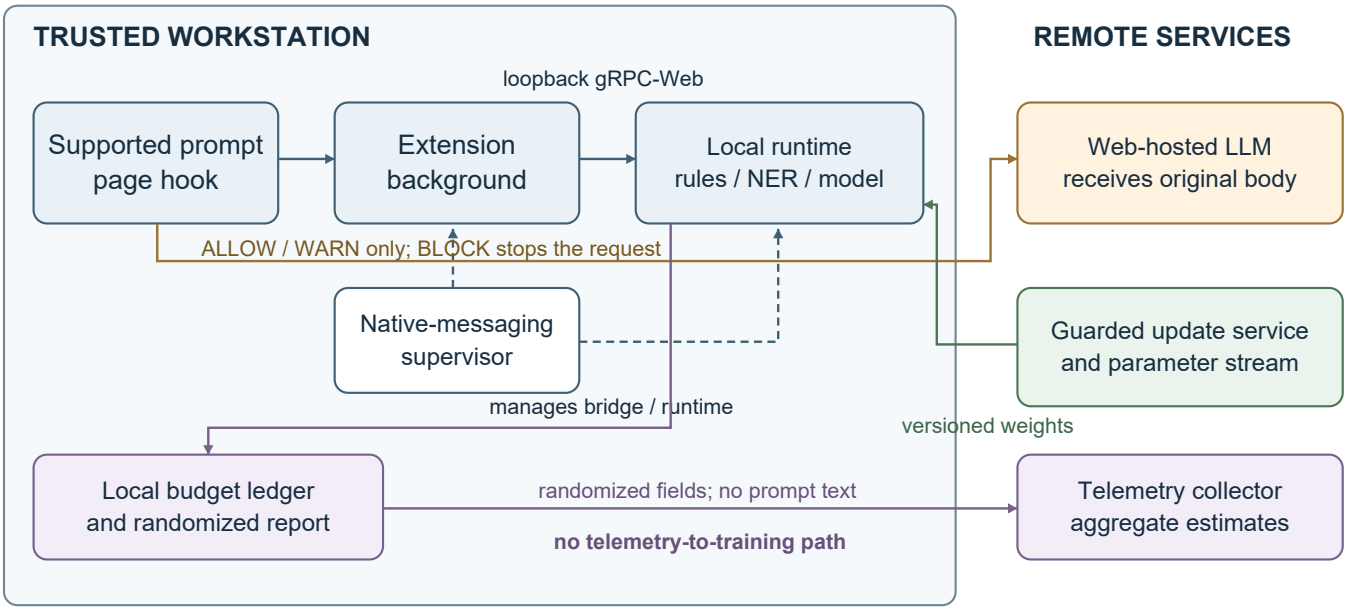


Fig. 1. PriVoke's three data paths. Supported prompt text is inspected inside the trusted workstation. Only an ALLOW or WARN decision releases the original body to the web-hosted LLM. Native messaging manages the local bridge/runtime; analysis uses loopback gRPC-Web. A separate update service supplies versioned weights, and a separate collector receives randomized event fields. Training consumes supplied labelled examples, not intercepted user prompts or telemetry. Dashed lines denote lifecycle control.

This design supports centralized training from supplied examples while keeping user inference local. It neither collects user gradients nor implements federated learning, and an implemented service path does not establish cloud deployment. RQ2 accordingly separates the ability to compute, publish and receive a revision from evidence that the particular revision should be retained.

D. Separate Randomized Telemetry

Telemetry describes coarse aspects of an event through five fields: action, an action/sensitivity risk bucket, one category selected by fixed precedence, a bounded release label and a four-hour time-of-day bin without a date. Their domain sizes are respectively $K_i = 3, 4, 11, 2, 6$; the category domain includes the no-category value. The workstation randomizes each field before release. Reports omit raw prompts, identifiers, exact confidence scores and per-layer findings, reducing the information presented to the collector without making the retained fields intrinsically harmless. The collector estimates event marginals and supplies no corrected labels or training inputs.

The intended local differential privacy guarantee limits how much the probability of an output can change between possible event contents [18]. Here the comparison holds the reporting opportunity and public configuration fixed; it does not hide whether an event was reported. An event budget ε is divided equally, giving each field $\varepsilon_i = \varepsilon/5$. A field with K_i possible values reports its true value with probability p_i and each alternative with probability q_i:

$$p_i = \frac{e^{\varepsilon_i}}{e^{\varepsilon_i} + K_i - 1}, \quad q_i = \frac{1}{e^{\varepsilon_i} + K_i - 1} \tag{1}$$

With independent ideal samples, multiplying the field likelihood ratios gives an event bound of e^{ε}, following composition [18]. The implemented sampler instead compares a fresh uniform 53-bit integer draw, $J/2^{53}$, with a binary64 threshold $\widetilde{p}_i$ and uses rejection sampling to choose an alternative. Its finite truth and alternative probabilities and event loss are

$$\begin{aligned} P_i &= \frac{\lceil 2^{53}\widetilde{p}_i \rceil}{2^{53}}, \quad Q_i = \frac{1 - P_i}{K_i - 1}, \\ \varepsilon_{\text{finite}} &= \sum_i \log \max\{P_i/Q_i, Q_i/P_i\}. \end{aligned} \tag{2}$$

This calculation conditions on the host arithmetic and independent uniform random bits; the ideal formula alone does not certify the implementation. A persistent ledger reserves budget before construction, with defaults of one per event and eight per UTC day. Construction or delivery failure does not refund a reservation. Retained state and a trusted clock are necessary for accounting. Report presence, contribution selection, delivery and network metadata remain outside the content guarantee. To estimate marginals, the collector inverts nominal response probabilities and clips counts to their feasible range. Whether those estimates remain useful after randomization and clipping is a separate empirical question.

TABLE I
RETAINED EVIDENCE AND ITS INFERENTIAL SCOPE. CONTEXTUAL TARGETS REMAIN PROVISIONAL RATHER THAN INDEPENDENTLY ADJUDICATED PRIVACY TRUTH.

RQ	Evidence	What it establishes
1	1,560 contextual cases; targeted ablation; 498 final presence rows	Agreement and detector contribution on declared targets.
1	33 installed receiver probes	Request disposition on tested paths.
2	Matched head/full continuation; 16-case guard	Paired gains/harms and conditional acceptance.
3	360 browser decisions; finite telemetry audit, simulation and ledger checks	Scoped cost, conditional content privacy and estimation/selection limits.

IV. EVALUATION METHOD

A. Questions and Evidence Units

The evaluation follows a decision from prediction to request disposition, then examines whether revised behavior survives retention and what the system consumes or reports. Table I identifies the evidence used for these questions. RQ1 combines a package comparison, targeted detector removal, a distinct final target and receiver probes; RQ2 uses one matched continuation study; RQ3 examines browser cost and the telemetry mechanism. Several analyses reuse the same cases, so agreement between them is not independent confirmation.

Detector execution is explicit throughout. Product calls request rules, NER and semantics, coded [2,3,4], and record legitimate skips after an early rule BLOCK. Semantic-only calls explicitly request [4] and assert that it alone actually executed. Requested layers therefore do not automatically count as executed layers, and a product outcome is not a semantic-only result.

Contextual targets and measures. The authored workload comprises 1,560 cases: 810 sensitive and 750 clean, with 260 per audience class, 35 scenario families and 24 audience-expression groups. Sensitivity, audience, category and allowed-action targets were prepared provisionally with assistant support and lack independent human adjudication. The cases were sealed for an earlier semantic comparison, then opened and reused for the present product comparison and ablation. These are development results on authored material, even where a subsequent protocol was fixed prospectively.

A nonzero sensitivity label or nonempty category set defines a positive binary prediction. Label-joint agreement requires exact sensitivity, audience and category-set matches; the *label-and-action joint* measure reported here additionally requires an allowed action. Allowed-action agreement is also measured alone. Under-restriction means an action below the allowed set, and over-restriction means an action above it. Actual receiver arrival is a separate systems observation. These definitions keep classification agreement, policy agreement and prevented transmission distinct.

Final annotation target. The once-only final endpoint contains 498 English PIIMB rows at revision `4a13e9f`: 236 annotation-positive and 262 annotation-negative [19]. Source inputs, all six packages and scoring were frozen before release; no tuning, reselection or promotion followed. The same nonzero-sensitivity-or-category projection is evaluated through sole-semantic execution. Source annotation presence differs from contextual authorization and from PIIMB's official character-masking/F2 task. Its 464 observed source-group keys describe declared relationships, not proven independent templates.

For paired uncertainty, the contextual analyses used 2,000 whole-cluster draws, seed 42 and linear 95% percentile ranges, separately by family and audience expression. Ablation expression groups were reconciled after collection to the intended audience grouping. Final-endpoint ranges used 2,000 paired whole-source-group draws, seed 42, row-weighted statistics and the same percentile convention. These descriptive, unadjusted ranges test sensitivity to declared dependencies; they do not establish crossed-cluster independence, unknown template separation or population coverage. Repeated calls and three training seeds do not enlarge the number of independent situations.

B. Comparison and Retention Protocols

Historical and updated efficient/balanced/quality packages saw the same contextual cases through the product handler, with rules and policy fixed. Updated packages descend from frozen-encoder head fitting, joint encoder/pooler/head continuation, and audience-supervised rehearsal. Checkpoints were selected on validation alone before the sealed comparison. Historical input windows were 64/96/128 tokens with category threshold 0.38, versus 512 WordPieces and threshold 0.5 for updated packages. Their parameter counts also differ: 0.019/0.037/0.054 versus 4.785/9.596/17.494 million. These native settings, tokenizers and training histories make the contrast a package comparison rather than an isolated effect of pretraining or architecture. First-pass outcomes are reported without treating repeatability passes as additional cases.

The targeted ablation fixes the updated balanced package, inputs and policy while comparing full [2,3,4], no-rules [3,4] and no-NER [2,4] requests. Actual execution and legitimate skips were verified for each. These contrasts ask whether the observed cascade errors depend on a particular detector; they cannot establish that removing it is safe on other workloads.

RQ2 instead fixes the initial model and training exposure. Head-only and full-model treatments each started from byte-identical official pretrained balanced encoder/pooler weights and zero task heads, rather than either package in RQ1. Seeds 42/43/44 received identical ordered 4,096 training exposures over 64 steps. Both minimized sensitivity and audience cross-entropy plus mean category binary cross-entropy, with dropout disabled. AdamW used head/encoder rates $10^{-3}/2 \times 10^{-5}$, zero weight decay, norm-one clipping and microbatch 4/effective batch 64. Full-model training enabled encoder opti-

mization and consequently changed the joint-gradient clipping scope as well.

Complete training/guard group and normalized-text exclusions removed 480 validation rows, leaving 1,080 reused provisional cases across 25 families: 570 sensitive and 510 clean. Every terminal candidate and the unchanged base underwent native in-process sole-semantic assessment; assessment performance selected no checkpoint. The 16 separate guard cases required improved joint correctness and zero action harms against a single derived target action per case. The broader assessment instead accepts its declared action sets. Passing the guard and improving assessment agreement therefore answer different questions. No candidate was published: effective-artifact mappings describe what the rule would retain, conditional on a small guard and a weak zero-head control.

C. Enforcement, Cost and Telemetry Protocols

The receiver study used the built unpacked extension in Linux Chromium, the standard native-host installer, native messaging, supervisor, bridge and historical balanced detector. A local HTTPS synthetic receiver recorded original bodies and checked for late sends. Thirty-three probes covered forwarding, blocking, faults, cancellation and recovery. Full-product and sole-semantic checks linked the expected action to actual execution outside the resource windows. This tests the installed request path, not an authenticated LLM provider or another platform.

Cost measurement used three fixed synthetic cases, one per action, over fetch and XHR. Two fresh profiles per transport/action cell gave twelve profiles, each with a first decision, five warmups and thirty measured rules/NER requests: 360 measured decisions. Page latency spans the analysis request to its result, excluding the receiver response. Nearest-rank percentiles pool only two profile clusters per cell. Each resource window begins before browser launch and ends after request/readiness checks, before profile teardown. Whole-container CPU and sampled charged memory include resident infrastructure, cache, driver, receiver and observer; charged memory is not process RSS/PSS [20]. Sampling requested 100-ms intervals and recorded actual gaps. There is no semantic processing, unprotected baseline or per-role attribution in this experiment.

The telemetry audit enumerated exact rational kernels for a fingerprinted Windows CPython 3.11.9 build and enclosed logarithms with outward 100-digit arithmetic, conditional on independent uniform OS bits. Budgets 0.5, 1 and 2 cover the three declared settings. Estimator simulation crossed those budgets with $n = 8, 80, 800, 8000$, all five field domains and balanced-integer, point-mass and minimum-one 1%-target truth histograms: 180 prespecified cells, each repeated 10,000 times with seed 20261014. NumPy 1.26.4/PCG64 samples binary64 approximations of finite probabilities; the collector's nominal inverse and independent $[0, n]$ clipping produce the estimates. Large n pools installation-days, each still capped at eight default reservations. The estimand is the fixed contributor histogram, excluding population-sampling variation. Ledger checks separately exercise persistence, concurrent admission and failure accounting. No deployment-usefulness threshold was specified.

TABLE II
FULL PRODUCT ON THE SAME PROVISIONAL CONTEXTUAL CASES. ENTRIES ARE HISTORICAL/UPDATED COUNTS. LABELS+ACTION DENOTES LABEL-AND-ACTION JOINT AGREEMENT; ACTION DENOTES ALLOWED-ACTION AGREEMENT (BOTH OUT OF 1,560). SENSITIVE AND CLEAN ARE CORRECTLY CLASSIFIED COUNTS OUT OF 810 AND 750, RESPECTIVELY. EVERY ARM HAD ZERO ERRORS.

Package	Labels+action	Action	Sensitive	Clean
Efficient	266/693	1029/1415	544/808	494/629
Balanced	160/717	1026/1409	771/802	318/631
Quality	226/879	1073/1421	598/810	468/641

V. RQ1: DETECTION AND ENFORCEMENT

A. Contextual Policy Agreement

The updated product packages more often selected the authored labels and an allowed action together. Label-and-action joint agreement increased by 27.37–41.86 percentage points, reaching 693–879 of 1,560 cases (Table II). All six runs completed without failed calls or layer errors. Efficient, balanced and quality gained 484/573/671 previously incorrect joint outcomes while losing 57/16/18 previously correct ones. Allowed-action gains were 389/389/348, with losses of 3/6/0. Quality therefore preserved every previously allowed-action match while losing eighteen label-and-action joint matches: preserving an intervention does not necessarily preserve its classification. These are improvements on reused provisional targets, and no package was promoted.

The execution records locate one important source of unnecessary restriction in the cascade. All updated packages over-restricted the same 139 cases: rules blocked 109 clean controls before later detectors could run and warned on thirty low-risk controls. Rules ran on every case and early-blocked 289 in total, leaving NER and semantics to execute on 1,271. The shared failures therefore could not be resolved simply by substituting a better semantic model behind those rules. NER contributed zero converted PriVoke findings after supported-label filtering. That observation concerns the findings admitted into the policy; it does not imply that the loaded recognizer was inactive or detected no raw entities.

Holding the balanced package and workload fixed allowed the detector-removal study to test this explanation. Removing rules raised allowed-action agreement from 1,409 to 1,530 and label-and-action joint agreement from 717 to 972. All 139 shared unnecessary restrictions were corrected, but eighteen previously correct actions were lost and under-restriction rose from twelve to thirty cases. The net gain of 121 actions thus came with additional missed protection under the provisional policy. Existing paired resampling placed the action-rate gain at 3.46–12.70 percentage points by family and 5.96–9.23 by audience expression. These separate descriptive ranges support the direction of the aggregate tradeoff without making the

TABLE III
FINAL SEMANTIC ANNOTATION PRESENCE: 236 POSITIVE/262 NEGATIVE ROWS. HISTORICAL/UPDATED COUNTS; TRUE/FALSE POSITIVES (TP/FP), TRUE/FALSE NEGATIVES (TN/FN). ALL SIX ARMS HAVE ZERO ERRORS.

Package	TP	TN	FP	FN
Efficient	140/218	61/45	201/217	96/18
Balanced	141/221	65/35	197/227	95/15
Quality	167/190	51/57	211/205	69/46

losses acceptable. Removing NER left full-product outputs unchanged, consistently with its zero converted findings on this fixture. Actual execution was verified for these contrasts; neither removal result establishes that a detector is unnecessary on other workloads.

B. Agreement on a Separate Annotation Target

The final annotation-presence endpoint tested whether the same packages distinguished a different set of positive and negative targets. All 2,988 semantic-only calls completed without errors and verified sole execution of the requested layer. Updated recall reached 80.51–93.64%, while specificity remained only 13.36–21.76% (Table III). Efficient and balanced lost specificity relative to their historical versions; quality's small increase remained uncertain. Frozen source-group resampling nevertheless gave paired net-accuracy gain ranges of 7.41–17.37, 5.35–15.03 and 1.02–10.71 percentage points, respectively. The apparently conflicting observations have a straightforward explanation: enough additional positives were recognized to improve average accuracy despite poor discrimination of annotation-negative rows.

The final result supplies a different check from the contextual product comparison. Here the target is source annotation presence and only the semantic layer runs; in the product comparison, provisional contextual labels and actions are assessed through a cascade. Their difference is not a controlled estimate of distribution shift or detector contribution. The low final specificity nevertheless establishes poor discrimination of annotation-negative rows under this declared projection. Neither endpoint supplies independently adjudicated truth about disclosure authorization.

C. Does a Decision Control Transmission?

Classification matters to enforcement only if the resulting decision changes what leaves the client. The completed installed-browser capture tested that connection across thirty-three declared correctness probes: the receiver observed eighteen unchanged-body sends and fifteen zero-send outcomes. Correlated runtime calls, receiver records and late-arrival checks supported those outcomes without a decision broker. Healthy full-pipeline and sole-semantic probes also verified actual execution, tying the observed request disposition to the tested detector path.

Fault probes showed why availability of a request path must be distinguished from preservation of its protection. Interrupting the parameter stream blocked semantic-only requests, whereas configured full-mode fallback ran rules and NER and forwarded the request. Restarting restored full execution; disabling protection or all layers forwarded requests by design. Fallback therefore maintained a reduced analysis path, but the capture does not establish equivalence to semantic protection. These completed observations remain separate from nine earlier failed captures, which were not pooled with them.

TABLE IV
MATCHED SEMANTIC-ONLY CONTINUATION ON 1,080 REUSED PROVISIONAL CASES. ENTRIES ARE HEAD-ONLY/FULL-MODEL COUNTS. LABELS+ACTION DENOTES LABEL-AND-ACTION JOINT AGREEMENT. THE UNCHANGED ZERO-HEAD BASE HAS ZERO SUCH MATCHES AND 480 ALLOWED-ACTION MATCHES. GUARD HARMS COMPARE EACH CANDIDATE WITH THAT BASE ON SIXTEEN SEPARATE CASES.

Seed	Labels+action	Allowed action	Guard harms
42	192/257	908/993	0/1
43	204/247	906/989	1/1
44	198/268	912/1003	0/1

Answer to RQ1. PriVoke improved agreement with provisional contextual policy and enforced the declared request contract on the tested installed paths. The evidence supports those two bounded achievements, while action errors, original-content forwarding under WARN and weak specificity on the separate annotation-presence task limit the effectiveness claim. Independent contextual accuracy, provider compatibility, prevention on every sensitive prompt and practical usability remain unestablished.

VI. RQ2: UPDATES AND PREVIOUSLY PROTECTED BEHAVIOR

A. Improvement Subject to Retention

A revision is useful only if its improvements survive the acceptance conditions required for publication. The RQ1 package comparison cannot isolate adaptation because capacity, tokenizer, threshold and supervision changed together. The matched continuation study instead fixes the base and training exposure while varying whether the encoder is optimized. This lets aggregate gains be examined alongside the previously correct cases each treatment loses. Its guard decisions assess candidate eligibility; they do not demonstrate publication or client refresh.

B. Matched Continuation and Guard Decisions

All six 64-step fits completed with finite parameters. Head-only training preserved every encoder tensor; full-model training changed 39 and recorded nonzero later encoder gradients. The comparison therefore exercised the intended difference in optimization, rather than merely different treatment labels. The six candidates and unchanged base then completed 7,560 native in-process semantic assessments without errors. Each of these seven snapshots saw the same 1,080 situations, so the call count measures completed assessment work rather than independent cases or deployed network operation.

Full-model continuation achieved more label-and-action joint matches and allowed-action matches than head-only

training in every seed (Table IV). For seeds 42/43/44, paired joint gains of 79/69/90 were offset by losses of fourteen, twenty-six and twenty cases; action gains of 123/124/110 were offset by thirty-eight, forty-one and nineteen losses. Full-model continuation also increased under-enforcement distance on 33/37/15 cases and over-enforcement distance on 17/13/17. The latter counts measure different directional changes and cannot be added into a mutually exclusive harm total. Together, these observations show why a higher aggregate score is insufficient evidence that previously correct protection has been retained. Three seeds on the same reused assessment do not establish the frequency of this tradeoff in an independent population.

The publication decision used a different comparison: each candidate against its unchanged base on the sixteen guard cases. Every full-model candidate introduced one action harm and failed that fixed rule, despite its better aggregate assessment. Head-only seeds 42/44 passed, while seed 43 failed. A failed candidate would therefore leave the base effective; it would not make the better assessment score available to clients. No candidate in this experiment was published or promoted, so these mappings describe hypothetical retention rather than deployed updates. The guard detected the losses represented by its cases and criteria, but a pass cannot rule out other harms or overcome the weak zero-head baseline.

Answer to RQ2. PriVoke can bind a revision to its base and reject candidates that violate the configured retention test. Useful improvement without sacrificed protection is not established: every matched full-model candidate improved aggregate agreement yet failed the guard, while head-only seed 43 also failed. The guard makes an acceptance boundary enforceable; the quality of supervision, coverage of retained cases and chosen criteria determine what protection that boundary can support.

VII. RQ3: Systems and Privacy Overhead

A. Measured Systems Cost

The browser measurements describe the cost of reaching a local rules/NER decision. Table V retains every transport/action cell rather than selecting the fastest path. All 240 ALLOW/WARN decisions ran both detectors; the 120 BLOCK decisions ended after rules and legitimately skipped NER. Differences between these cells therefore include different amounts of work. Their medians range from 27.9 to 35.0 ms, while the observed maxima describe sixty requests in two profiles per cell rather than an independently established tail guarantee. The measurement ends at the analysis result and includes neither downstream completion nor semantic processing.

The resource windows covered 126.40 seconds and accumulated 193.29 CPU-seconds across the container, including 0.69 observer CPU-seconds. CPU-seconds sum concurrent processor use and are distinct from elapsed time. Maximum sampled charged memory was 1,267.91 MiB, including resident services and cache as well as the driver and receiver. The maximum sampling gap of 622.84 ms leaves shorter spikes potentially unobserved. Final-read and cleanup checks passed, but neither isolates the extension and detector from measured infrastructure. Without an unprotected baseline or per-role attribution, these values cannot establish incremental PriVoke overhead.

TABLE V
Page-observed rules/NER decision latency (ms). Each cell contains sixty measurements in two fresh profiles. Max is the observed maximum. Receiver response and semantic processing are excluded.

Transport	Action	p50	p95	Max
fetch	ALLOW	35.0	62.5	73.6
fetch	WARN	31.9	36.1	59.4
fetch	BLOCK	27.9	31.6	33.2
XHR	ALLOW	30.8	35.8	57.7
XHR	WARN	31.3	34.6	59.2
XHR	BLOCK	28.5	32.2	34.1

TABLE VI
Illustrative prespecified category cells ($K = 11$, event $\varepsilon = 1$), each with 10,000 repetitions. Truth, clipped bias and RMSE concern the first category in count units. Clip is its clipping frequency; Sum is the frequency that independently clipped marginals do not sum to n (tolerance 10^{-9}).

n	Truth	Bias	RMSE	Clip (%)	Sum (%)
80	7	+22.84	42.32	85.62	99.99
8000	727	+237.50	1028.47	28.32	99.78

B. Privacy Cost and Telemetry Utility

Auditing the finite implementation changed the privacy statement that could be made from the ideal formula alone. On the fingerprinted host, maximum event privacy loss exceeded nominal budgets 0.5, 1 and 2 by approximately $(1.294, 1.503, 1.226) \times 10^{-15}$. The excess is numerically tiny, but an exact nominal guarantee does not hold for that kernel. Eight default reservations compose to approximately $8 + 1.202 \times 10^{-14}$ under the same independent uniform-entropy assumptions. This bounds admitted event content under the audited conditions, without certifying other builds or deployment entropy.

The estimator poses a different question: whether useful counts can be recovered after randomization. Table VI illustrates two prespecified cells already reported in the original simulation: balanced-integer category histograms at event budget one and $n = 80/8000$. These examples are not a utility acceptance test or a claim that other cells perform similarly. They expose substantial clipping bias and RMSE at both scales. Absolute error rises with contributor count even though relative error can fall, and independently estimated marginals almost never sum to the known total. Thus a larger collection alone does not make these outputs reliable counts of the synthetic contributors. Deployment accuracy and an acceptable application-specific error remain unmeasured.

Accounting checks admitted eight default reservations and denied the ninth, retained the cap after process reopen and

reset it on the next UTC day. Clock rollback and corrupt state failed closed; sixteen of forty concurrent epsilon-0.5 requests were admitted. Construction, queue and delivery failures still consumed their reservations. These outcomes support bounded admission under the tested state and clock conditions, but admission also selects which events appear. A constructed 32-event sequence contains 25% BLOCK overall; reversing its order changes BLOCK representation among the first eight contributors from 100% to 0%. Content randomization cannot repair this selection effect or conceal participation. A content-privacy bound consequently does not establish representative monitoring.

Answer to RQ3. The tested configuration has measured local decision and infrastructure costs, while incremental overhead and semantic-pipeline cost remain unmeasured. Telemetry has an audited conditional event-content privacy loss slightly above its nominal budget, substantial synthetic estimation error and a separate contribution-selection effect. These findings quantify limits of the implemented reporting path without establishing useful deployment monitoring or protection of participation and network metadata.

VIII. Discussion and Limitations

A. What the System Evidence Supports

Privacy protection depends on how decisions are connected as well as how accurately a model classifies a prompt. An early rule BLOCK prevents semantic contribution, a policy-correct WARN releases original content, and an improved candidate remains unavailable if it fails retention. PriVoke makes these dependencies inspectable through execution records, version-bound candidates and separate reporting. Its contribution is this evaluated lifecycle, which exposes where detection, request handling and revision acceptance diverge without treating one successful component as proof of overall protection.

Local sanitization clarifies the practical question left open. Casper couples acknowledged warnings with restoration, HaS recovers hidden response entities, and ProSan evaluates utility after selective replacement [8], [10], [11]. These functions address task continuation after intervention. PriVoke neither rewrites prompts nor restores answers, so its request contract cannot establish the same benefit. An extension would need to evaluate authorized task completion directly.

B. Ground Truth and Generalization Remain Open

Contextual integrity concerns appropriate information flows [6]; entity recognition supplies only part of that judgment. Independent contextual ground truth remains the largest scientific gap because PriVoke's sensitivity, audience, category and action targets are assistant-provisional. Neither agreement with them nor agreement with annotation-presence labels establishes authorization or human agreement. The package gains on authored contextual targets and poor final annotation-negative specificity both matter, while their different targets and execution scopes prevent a direct causal comparison. Entity identification, disclosure appropriateness, intervention choice and task utility each require suitable evidence.

Source-group keys and lexical exclusions track declared relationships without certifying template independence, covering paraphrases or removing unknown upstream pretraining exposure. The final endpoint is unevenly distributed across 308 Nemotron, 89 AI4privacy, eighty Gretel, twenty Privy and one MAPA row. These English synthetic or machine-generated workloads do not represent other languages or organizational policies. Cluster sensitivities remain descriptive and unadjusted; repeated runs and three seeds are not independent populations, and native-package differences constrain causal attribution. Learning from the opened final endpoint requires a new protected successor for confirmation.

C. Retention, Availability and Practical Utility

The gains and losses illustrate the update problem studied by positive-congruent training: higher average accuracy can coexist with newly incorrect cases [7]. PriVoke's acceptance guard addresses this problem only within its cases and criteria, a limitation amplified by its small size and weak zero-head control. Rejected candidates supply no served improvement. Stronger evaluation requires common pinned bases, independently reviewed retention cases and explicit casewise harm criteria, with failed candidates kept distinct from effective artifacts.

Blocking unavailable analysis can prevent disclosure while interrupting an authorized task; WARN instead proceeds without informed acknowledgement. Heuristic confidence is not calibrated joint correctness. The installed Linux evidence covers declared Chromium/native receiver paths with the historical detector, leaving authenticated providers, Windows, updated-model installation, workers, attachments and other transports unverified. Authorized task completion, unnecessary-intervention cost, warning comprehension and reformulation effort were unmeasured, preventing a claim of practical utility or human benefit.

D. Measurement Boundaries and Next Steps

Container infrastructure and instrumentation, together with rules/NER-only timings, leave incremental client and semantic cost unresolved. RAPPOR establishes precedent for accounting for repeated randomized contributions [13]; PriVoke's ledger relies on retained state and a trusted clock, with its finite calculation also assuming independent uniform OS bits. Participation, delivery selection and metadata remain outside the content bound. No deployment-usefulness threshold was defined, and randomized aggregates provide no corrected contextual training labels. Reporting privacy, representative monitoring and useful supervision thus remain separate requirements.

Further evaluation should target the missing outcomes rather than accumulate repeated agreement on the same provisional cases. Independent contextual adjudication and a new protected endpoint would test whether policy decisions generalize. Broader receiver coverage, matched unprotected and semantic-processing cost measurements, and user or task outcomes

would then connect protection to its practical costs and benefits. These are empirical requirements that the present mechanism and bounded experiments cannot resolve by themselves.

IX. Conclusion

PRIVOKE connects local prompt inspection, explicit request enforcement, version-bound guarded revision and separate randomized reporting in an implemented lifecycle. Its evaluation shows why those mechanisms need distinct acceptance evidence. Authored contextual agreement improved, yet action errors and poor specificity on the separate annotation-presence target remained. Installed-browser probes verified supported request handling, including original-content forwarding under WARN. Every matched full-model candidate improved aggregate agreement but failed retention, making average gain an insufficient basis for publication. Resource and telemetry studies quantified bounded costs and privacy behavior without demonstrating deployment utility. The contribution is an inspectable mechanism and evidence of its limits; independent contextual accuracy, useful adaptation without lost protection and human benefit remain open.

AI Assistance

AI assistance was used to produce provisional contextual labels and support manuscript drafting. GPT-6 Astra assisted manuscript review and rewriting. The provisional labels have not been established as independently adjudicated ground truth. Responsibility for the manuscript and its claims remains with the authors.

References

[1] B. B. Gupta, G. Akshat, V. Arya, A. Wadee, D. Alsalman, and P. Vijayakumar, "Enhancing user prompt confidentiality in large language models through advanced differential encryption," *Computers & Electrical Engineering*, vol. 116, pp. 109215–109215, 02 2024.

[2] W. Liang, Y. Zhang, M. Codreanu, J. Wang, H. Cao, and J. Zou, "The widespread adoption of large language model-assisted writing across society," *Patterns*, vol. 6, no. 12, p. 101366, 2025.

[3] T. Koraag, N. Wagner, F. Dobslaw, and L. Gren, "Comparing open-source and commercial llms for domain-specific analysis and reporting: Software engineering challenges and design trade-offs," *arXiv preprint arXiv:2509.24344*, 2025.

[4] S. Ray, "Samsung bans chatgpt among employees after sensitive code leak," *Forbes Business*, 05 2023.

[5] J. Dastin and A. Tong, "Focus: Google, one of ai's biggest backers, warns own staff about chatbots," *Reuters (Technology)*, 06 2023.

[6] H. Nissenbaum, "Privacy as contextual integrity," *Washington Law Review*, vol. 79, no. 1, pp. 119–158, 2004.

[7] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14299–14308, 2021.

[8] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," 08 2024. Version 1, 13 August 2024; supplied 13-page PDF. Not the 2025 proceedings version.

[9] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," in *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, pp. 122–131, 2025. Related preprint: arXiv:2408.07004v1.

[10] Y. Chen, T. Li, H. Liu, and Y. Yu, "Hide and seek (has): A lightweight framework for prompt privacy protection," 09 2023.

[11] Z. Shen, Z. Xi, Y. He, W. Tong, J. Hua, and S. Zhong, "The fire thief is also the keeper: Balancing usability and privacy in prompts," 06 2024.

[12] E. Keles and S. Bahtiyar, "Promptshield: Policy-aware dlp framework for generative ai prompts," in *2025 10th International Conference on Computer Science and Engineering (UBMK)*, pp. 1537–1542, 2025. Distinct conference publication record; relationship to the 2026 thesis not established.

[13] Ú. Erlingsson, V. Pihur, and A. Korolova, "RAPPOR: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014.

[14] "BERT Miniatures: google/bert_uncased_L-4_H-128_A-2 model card." Pinned revision c29bee83fc7f003ac8c5e6e135529da4ecddb7c3; Apache-2.0 declaration; accessed October 12, 2026.

[15] "BERT Miniatures: google/bert_uncased_L-2_H-256_A-4 model card." Pinned revision 4e937a8675e5afd9a4836735c186ec01695bc3ea; Apache-2.0 declaration; accessed October 12, 2026.

[16] "BERT Miniatures: google/bert_uncased_L-12_H-256_A-4 model card." Pinned revision 2ab9a0f41435a4d23d4cbc11cc3e7c922545f13d; Apache-2.0 declaration; accessed October 12, 2026.

[17] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," 2019.

[18] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[19] "PIIMB: PII Masking Benchmark." Pinned Hugging Face dataset card. Revision 4a13e9f; accessed 14 October 2026. Character-level zero-shot masking benchmark; CC BY-NC 4.0.

[20] T. Heo, "Control Group v2." The Linux Kernel documentation. https://docs.kernel.org/admin-guide/cgroup-v2.html. CPU and Memory interface definitions; accessed 15 October 2026.